



**Bharat Ratna Indira Gandhi College of
Engineering, Solapur
Master Of Computer Application**

**Subject: Data Structure and Algorithms
(MCAC102)**

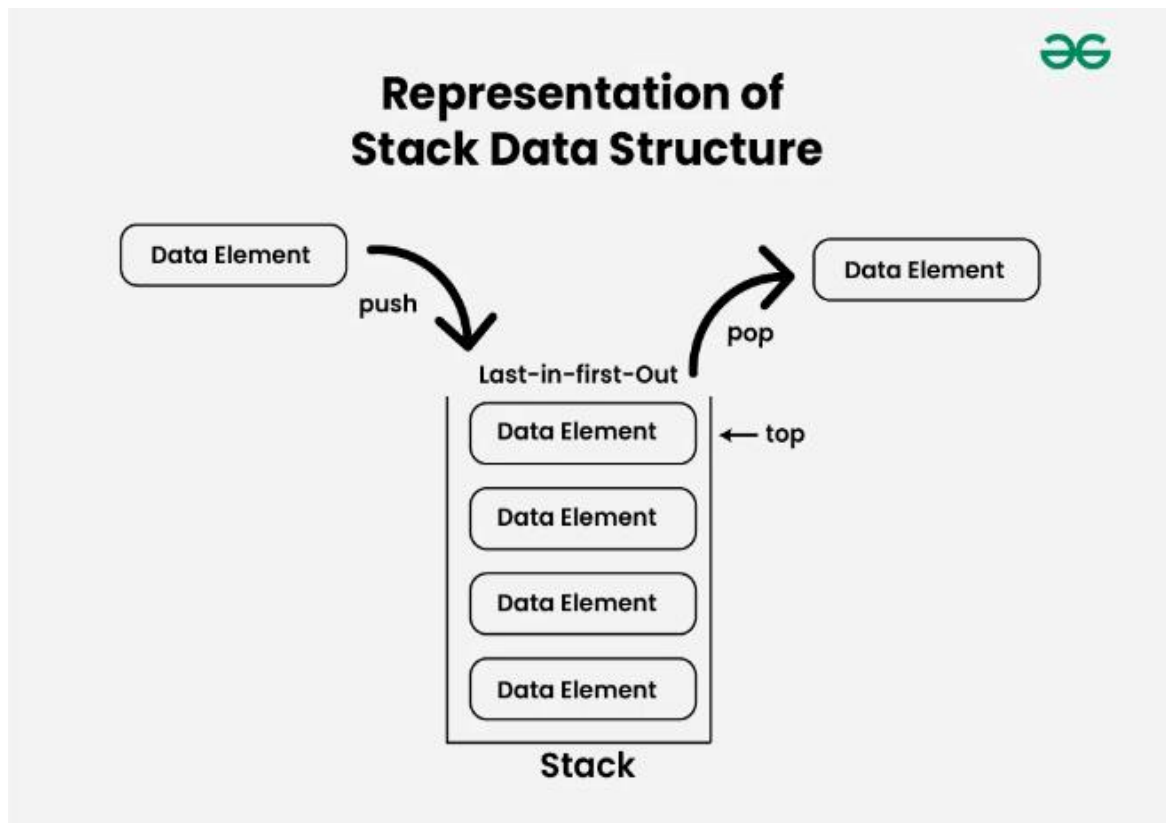
**Semester-I
Unit 2: Stack and Queues**

Unit 02: Stack and Queues

A. Stacks - Concept

Concept:

- **Definition:** A stack is a linear data structure that follows the Last In, First Out (LIFO) principle. This means the last element added to the stack is the first one to be removed. It's analogous to a stack of plates where you add and remove plates from the top.



- **Key Operations on Stack Data Structures**

Push: Adds an element to the top of the stack.

Pop: Removes the top element from the stack.

Peek: Returns the top element without removing it.

IsEmpty: Checks if the stack is empty.

IsFull: Checks if the stack is full (in case of fixed-size arrays).

- **Applications of Stack Data Structures**

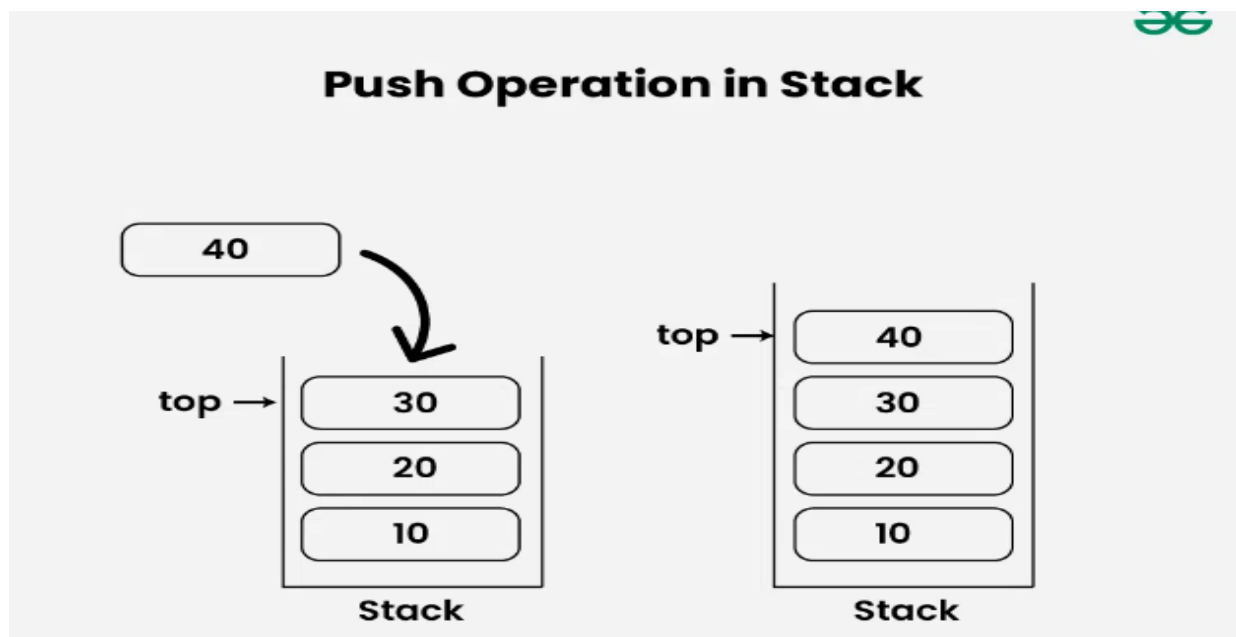
- a. Recursion
- b. Expression Evaluation and Parsing
- c. Depth-First Search (DFS)
- d. Undo/Redo Operations
- e. Browser History
- f. Function Calls

A. Push Operation in Stack Data Structure:

Adds an item to the stack. If the stack is full, then it is said to be an **Overflow condition**.

Algorithm for Push Operation:

- I. Before pushing the element to the stack, we check if the stack is **full**.
- II. If the stack is full (**top == capacity-1**), then **Stack Overflows** and we cannot insert the element to the stack.
- III. Otherwise, we increment the value of top by 1 (**top = top + 1**) and the new value is inserted at **top position**.
- IV. The elements can be pushed into the stack till we reach the **capacity** of the stack.

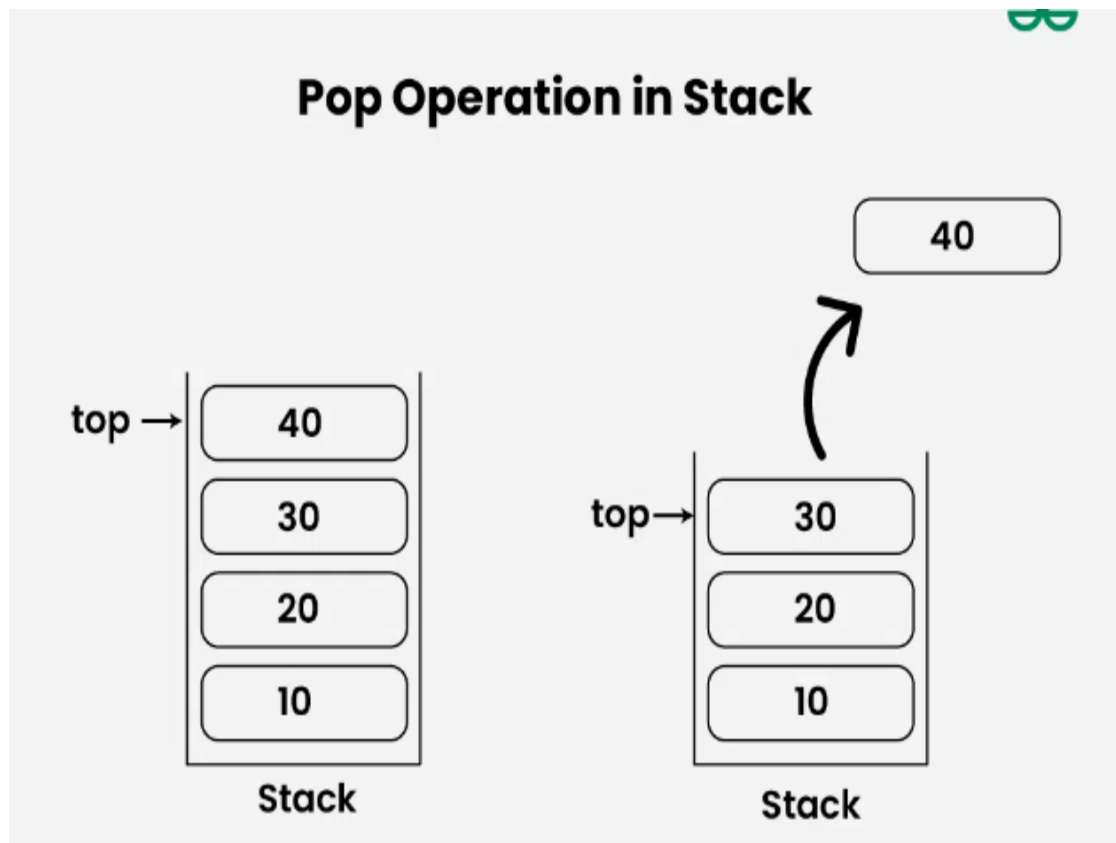


B. Pop Operation in Stack Data Structure:

Removes an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an **Underflow condition**.

Algorithm for Pop Operation:

- Before popping the element from the stack, we check if the stack is **empty**.
- If the stack is empty ($\text{top} == -1$), then **Stack Underflows** and we cannot remove any element from the stack.
- Otherwise, we store the value at top, decrement the value of top by 1 ($\text{top} = \text{top} - 1$) and return the stored top value.

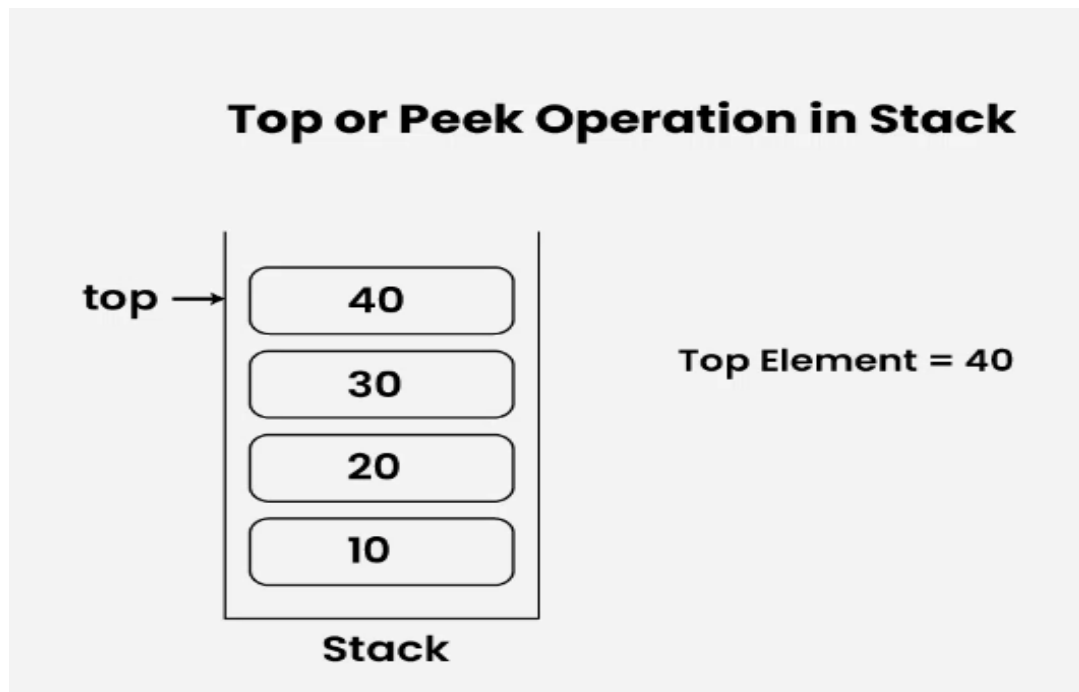


C. Top or Peek Operation in Stack Data Structure:

Returns the top element of the stack.

Algorithm for Top Operation:

- Before returning the top element from the stack, we check if the stack is empty.
- If the stack is empty ($\text{top} == -1$), we simply print "Stack is empty".
- Otherwise, we return the element stored at **index = top**.

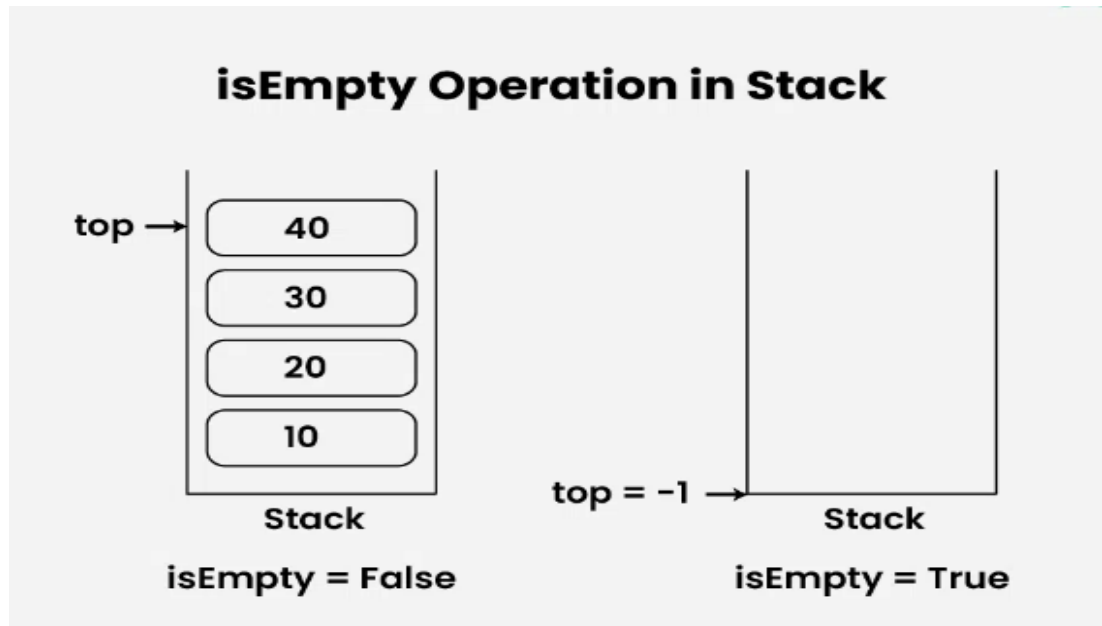


D. isEmpty Operation in Stack Data Structure:

Returns true if the stack is empty, else false.

Algorithm for isEmpty Operation:

- Check for the value of **top** in stack.
- If ($\text{top} == -1$), then the stack is **empty** so return **true**.
- Otherwise, the stack is not empty so return **false**.

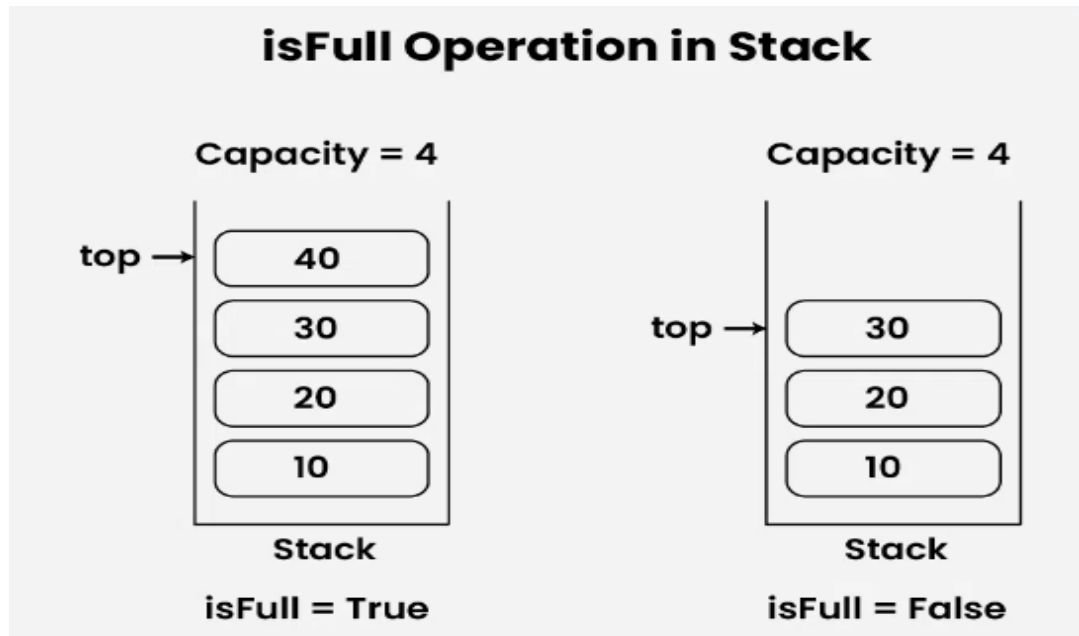


E. isFull Operation in Stack Data Structure:

Returns true if the stack is full, else false.

Algorithm for isFull Operation:

- Check for the value of **top** in stack.
- If (**top == capacity-1**), then the stack is **full** so return **true**.
- Otherwise, the stack is not full so return **false**.



- **Example:**

- **Imagine:** A stack of books where you add books on top and remove them from the top.
- **Operations in Stack:**
 1. **Push Operation:** Place a new book on the stack.
 2. **Pop Operation:** Remove the topmost book from the stack.
 3. **Peek Operation:** Look at the topmost book without removing it.

Visual Representation:

- **Initial State:**

[Bottom]

[Book1]

[Book2]

[Book3] <-- Top

- **After Push (Book4):**

[Bottom]

[Book1]

[Book2]

[Book3]

[Book4] <-- Top

- **After Pop ():**

[Bottom]

[Book1]

[Book2]

[Book3] <-- Top

B. Primitive Operations in Stack

1. Push Operation

Description:

- The **Push** operation adds an element to the top of the stack. If the stack is implemented using an array, it involves incrementing the index of the top and inserting the new element there.

Example Implementation [Array Representation]:

```
#define MAX 100
```

```
int stack [MAX];
```

```
int top = -1; // Initialize the top to -1 indicating an empty stack
```


Sem-I[DSA]-Unit-02

```
void push (int element) {  
    if (top >= MAX - 1) {  
        printf("Stack Overflow\n");  
    } else {  
        stack[++top] = element;  
    }  
}
```

Illustration:

1. Initial State:

[Empty]

2. After Push (5):

[Top]

[5]

3. After Push (10):

[Top]

[10]

[5]

4. After Push (15):

[Top]

[15]

[10]

[5]

Details:

- The top index starts at -1 for an empty stack.
 - After each push, top is incremented, and the element is placed at the new top index.
-

2. Pop Operation

Description:

- The **Pop** operation removes the element from the top of the stack and returns it. If the stack is empty, it results in underflow.

Example Implementation:

Array Representation:

```
int pop() {  
    if (top < 0) {  
        printf("Stack Underflow\n");  
        return -1; // Indicate an error  
    } else {  
        return stack[top--];  
    }  
}
```

Illustration:

1. **Initial State:**

[Top]

[15]

[10]

[5]

2. After Pop():

- **Returned Value:** 15
- **New State:**

[Top]

[10]

[5]

Details:

- The pop operation checks if top is less than 0 to determine if the stack is empty.
 - It returns the element at the top index and then decrements top.
-

3. Peek (or Top) Operation

Description:

- The **Peek** operation retrieves the top element of the stack without removing it. This operation allows you to view the topmost element.

Example Implementation:

Array Representation:

```
int peek() {  
    if (top < 0) {  
        printf("Stack is Empty\n");  
        return -1; // Indicate an error  
    } else {
```

```
        return stack[top];  
    }  
}
```

Illustration:

1. **Initial State:**

[Top]

[10]

[5]

2. **Peek():**

- **Returned Value:** 10
- **Stack State Remains Unchanged:**

[Top]

[10]

[5]

Details:

- The peek operation checks if the stack is empty by verifying if top is less than 0.
- It returns the element at the top index without modifying the stack.

4. IsEmpty Operation

Description:

- The **IsEmpty** operation checks if the stack has any elements. It returns true if the stack is empty and false otherwise.

Example Implementation:

Array Representation:

```
bool isEmpty() {  
    return top < 0;  
}
```

Illustration:

1. **Stack State:**

[Empty]

2. **IsEmpty():**

- **Returned Value:** true

3. **After Push(5):**

[Top]

[5]

4. **IsEmpty():**

- **Returned Value:** false

Details:

- The isEmpty function checks if the top index is less than 0 to determine if the stack is empty.
-

C. Stack Abstract Data Type (ADT)

Definition:

- The **Stack Abstract Data Type (ADT)** is a conceptual model that defines a stack's behavior without specifying its concrete implementation. It focuses on the operations a stack supports and their intended use rather than how these operations are implemented.

Core Operations:

1. **Push:** Adds an element to the top of the stack.
2. **Pop:** Removes the top element from the stack.
3. **Peek/Top:** Retrieves the top element without removing it.
4. **IsEmpty:** Checks if the stack is empty.

Properties:

- **Last In, First Out (LIFO):** The most recently added element is the first one to be removed.
- **No Direct Access:** Elements can only be accessed from the top of the stack.

Example of Stack ADT

- **Push(item):** Adds item to the top of the stack.
- **Pop():** Removes and returns the top item of the stack.
- **Peek():** Returns the top item without removing it.
- **IsEmpty():** Returns true if the stack is empty, otherwise false.

Sem-I[DSA]-Unit-02

```
#include <iostream>

using namespace std;

#define MAX 100 // Define the maximum size of the stack

class Stack {

private:

    int arr[MAX]; // Array to hold stack elements

    int top;    // Index of the top element

public:

    // Constructor

    Stack() {

        top = -1; // Initialize top to -1 indicating an empty stack

    }

    // Push operation

    void push(int item) {

        if (top >= MAX - 1) {

            cout << "Stack Overflow\n";

        } else {

            arr[++top] = item; // Increment top and add item to stack

            cout << item << " pushed to stack\n";

        }

    }

    // Pop operation

    int pop() {
```

```
        if (top < 0) {  
            cout << "Stack Underflow\n";  
            return -1; // Indicate an error  
        } else {  
            return arr[top--]; // Return top item and decrement top  
        }  
    }  
}  
  
// Peek operation  
  
int peek() {  
    if (top < 0) {  
        cout << "Stack is Empty\n";  
        return -1; // Indicate an error  
    } else {  
        return arr[top]; // Return top item  
    }  
}  
  
// IsEmpty operation  
  
bool isEmpty() {  
    return top < 0; // Return true if top is less than 0 (empty stack)  
}  
  
};  
  
int main() {  
    Stack s; // Create a stack object
```



```
s.push(10); // Push 10

s.push(20); // Push 20

s.push(30); // Push 30

cout << s.peek() << " is at the top of the stack\n"; // Peek the top item

cout << s.pop() << " popped from stack\n"; // Pop the top item

cout << s.pop() << " popped from stack\n"; // Pop another item

cout << (s.isEmpty() ? "Stack is Empty" : "Stack is not Empty") << endl; // Check if
empty

return 0;

}
```

Output :

- 10 pushed to stack
- 20 pushed to stack
- 30 pushed to stack
- 30 is at the top of the stack
- 30 popped from stack
- 20 popped from stack
- Stack is not Empty

D. Representation of Stacks Using Sequential Organization

In sequential organization, a stack is implemented using an array. This method involves using a fixed-size array to store stack elements and a variable to track the index of the top element. This approach is straightforward and efficient, but it has limitations such as fixed size and potential overflow issues.

Detailed Explanation:

1. Array Representation

- **Definition:** A stack is represented using an array where elements are pushed and popped from the end of the array, denoted by the top index.
- **Components:**
 - **Array:** Stores the stack elements.
 - **Top:** An index variable that points to the top of the stack.

2. Operations:

- **Push:** Add an element to the top of the stack.
- **Pop:** Remove the element from the top of the stack.
- **Peek:** Retrieve the top element without removing it.
- **IsEmpty:** Check if the stack is empty.

```
#include <iostream>

using namespace std;

#define MAX 100 // Maximum size of the stack

class Stack {

    int stack[MAX];

    int top;

public:

    Stack() { top = -1; } // Initialize stack to empty

    void push(int element) {

        if (top >= MAX - 1) {

            cout << "Stack Overflow" << endl;
```

```
    } else {  
        stack[++top] = element;  
        cout << element << " pushed to stack" << endl;  
    }  
}  
  
int pop() {  
    if (top < 0) {  
        cout << "Stack Underflow" << endl;  
        return -1; // Indicate an error  
    } else {  
        return stack[top--];  
    }  
}  
  
int peek() {  
    if (top < 0) {  
        cout << "Stack is Empty" << endl;  
        return -1; // Indicate an error  
    } else {  
        return stack[top];  
    }  
}  
  
bool isEmpty() {  
    return top < 0;
```

```
    }  
};  
  
int main() {  
    Stack s;  
  
    s.push(10);  
  
    s.push(20);  
  
    s.push(30);  
  
    s.push(40);  
  
    cout << s.peek() << " is at the top of the stack" << endl;  
  
    cout << s.pop() << " popped from stack" << endl;  
  
    cout << s.pop() << " popped from stack" << endl;  
  
    cout << s.peek() << " is now at the top of the stack" << endl;  
  
    return 0;  
}
```

E. Applications of Stacks: Expression Evaluation and Conversion

1. Expression Evaluation

1.1. Postfix Expression Evaluation (Reverse Polish Notation - RPN)

In postfix notation, operators follow their operands. This notation eliminates the need for parentheses to denote operation precedence. To evaluate a postfix expression, use a stack:

Algorithm:

1. **Initialize** an empty stack.
2. **Read** the postfix expression from left to right.

3. **For each token:**

- **If it's an operand**, push it onto the stack.
- **If it's an operator** (e.g., +, -, *, /):
 - Pop the top two operands from the stack.
 - Apply the operator to these operands.
 - Push the result back onto the stack.

4. **At the end**, the stack should contain one value, which is the result of the postfix expression.

Example:

Expression: 5 6 2 + * 12 4 / -

1. Push 5 → Stack: [5]
2. Push 6 → Stack: [5, 6]
3. Push 2 → Stack: [5, 6, 2]
4. Encounter +:
 - Pop 2 and 6
 - Compute $6 + 2 = 8$
 - Push 8 → Stack: [5, 8]
5. Encounter *:
 - Pop 8 and 5
 - Compute $5 * 8 = 40$
 - Push 40 → Stack: [40]
6. Push 12 → Stack: [40, 12]
7. Push 4 → Stack: [40, 12, 4]

8. Encounter /:

- Pop 4 and 12
- Compute $12 / 4 = 3$
- Push 3 $\rightarrow$ Stack: [40, 3]

9. Encounter -:

- Pop 3 and 40
- Compute $40 - 3 = 37$
- Push 37 $\rightarrow$ Stack: [37]

Result: 37

1.2. Prefix Expression Evaluation

Prefix notation (Polish notation) places operators before their operands. To evaluate a prefix expression:

Algorithm:

1. **Reverse** the given prefix expression.
2. **Use** a stack to evaluate the reversed expression as if it were a postfix expression.

Example:

Expression: - * 5 + 6 2 / 12 4

1. Reverse the expression to get 4 12 / 2 6 + 5 * -
2. Evaluate the reversed expression as a postfix:
 - Push 4, 12 $\rightarrow$ Stack: [4, 12]
 - Encounter /:
 - Pop 12, 4
 - Compute $4 / 12 = 0.333$ (or integer division 0)

- Push 0.333 (or 0) $\rightarrow$ Stack: [0.333]
 - Push 2, 6 $\rightarrow$ Stack: [0.333, 2, 6]
 - Encounter +:
 - Pop 6, 2
 - Compute $2 + 6 = 8$
 - Push 8 $\rightarrow$ Stack: [0.333, 8]
 - Encounter *:
 - Pop 8, 0.333
 - Compute $0.333 * 8 = 2.666$
 - Push 2.666 $\rightarrow$ Stack: [2.666]
 - Encounter -:
 - Pop 2.666
 - Result: 2.666
-

2. Expression Conversion

2.1. Infix to Postfix Conversion

Infix expressions (e.g., $A + B$) require parentheses to specify operator precedence. Conversion to postfix involves the following steps:

Algorithm:

1. **Initialize** an empty stack for operators and an empty list for the output.
2. **Read** the infix expression from left to right.
3. **For each token:**
 - **If it's an operand**, add it to the output list.

- **If it's an operator:**
 - Pop operators from the stack to the output list while the operators on the stack have higher or equal precedence than the current operator.
 - Push the current operator onto the stack.
 - **If it's a parenthesis:**
 - **Left parenthesis:** Push onto the stack.
 - **Right parenthesis:** Pop from the stack to the output list until a left parenthesis is encountered.
4. **Pop** any remaining operators from the stack to the output list.

Example:

Expression: $A * (B + C) - D$

1. Initialize stacks and output list: Stack: [], Output: []
2. Process A $\rightarrow$ Output: [A]
3. Process * $\rightarrow$ Stack: [*]
4. Process ($\rightarrow$ Stack: [* (]
5. Process B $\rightarrow$ Output: [A, B]
6. Process + $\rightarrow$ Stack: [* (+]
7. Process C $\rightarrow$ Output: [A, B, C]
8. Process):
 - Pop from stack to output until (: Output: [A, B, C, +], Stack: [*]
9. Process -:
 - Pop from stack to output: Output: [A, B, C, +, *], Stack: [-]
10. Process D $\rightarrow$ Output: [A, B, C, +, *, D]

Postfix Expression: A B C + * D -

2.2. Infix to Prefix Conversion

Prefix notation requires operators before operands. The conversion can be done using a similar process as infix to postfix but with steps reversed:

Algorithm:

1. **Reverse** the infix expression.
2. **Replace** (with) and vice versa.
3. **Convert** the modified expression to postfix.
4. **Reverse** the resulting postfix expression to get the prefix expression.

Example:

Expression: A * (B + C) - D

1. Reverse: D - (C + B) * A
2. Replace parentheses: D -) B + C (* A
3. Convert to postfix: D B C + A * -
4. Reverse: - * A + C B D

Prefix Expression: - * A + B C D

F. Queue

What is Queue in Data Structures?

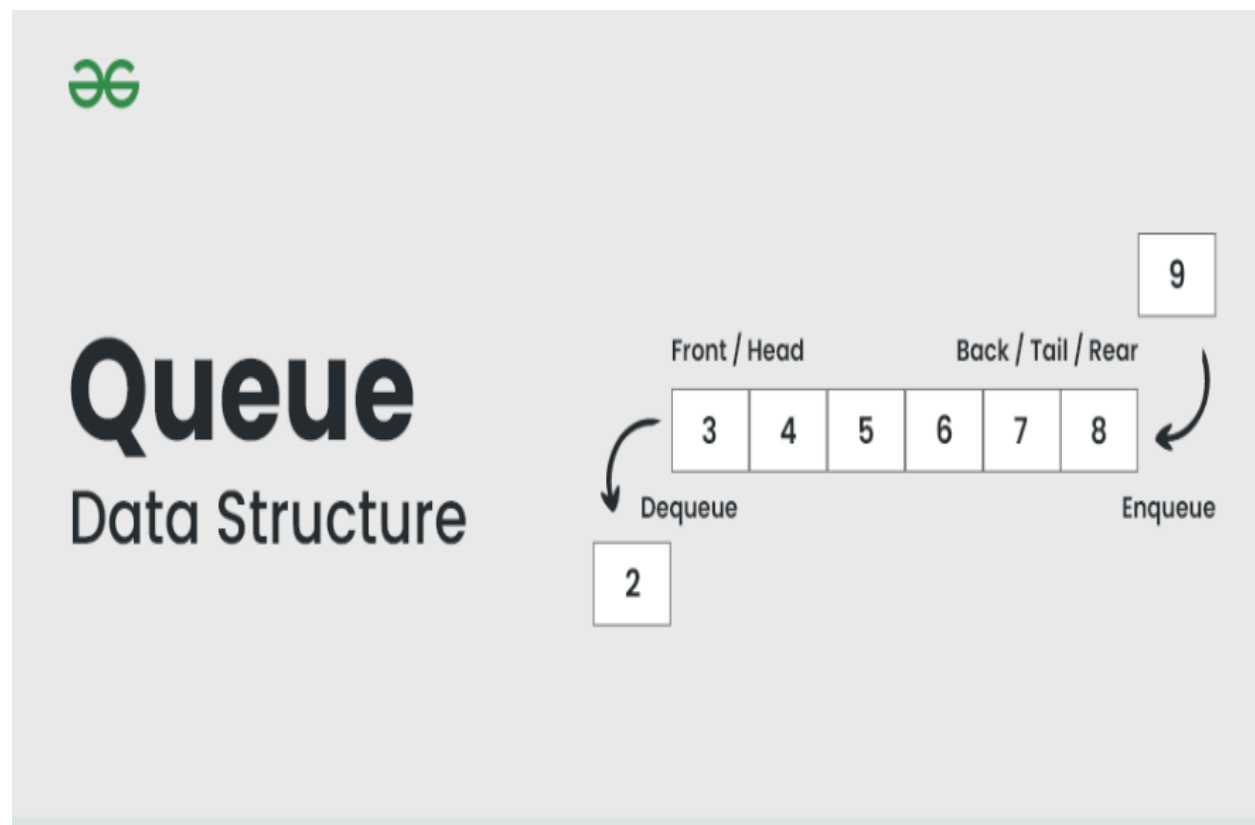
A queue is a linear data structure that follows the **First-In-First-Out (FIFO)** principle. It operates like a line where elements are added at one end (**rear**) and removed from the other end (**front**).

Basic Operations of Queue Data Structure

- **Enqueue (Insert):** Adds an element to the rear of the queue.
- **Dequeue (Delete):** Removes and returns the element from the front of the queue.
- **Peek:** Returns the element at the front of the queue without removing it.
- **Empty:** Checks if the queue is empty.
- **Full:** Checks if the queue is full.

Applications of Queue

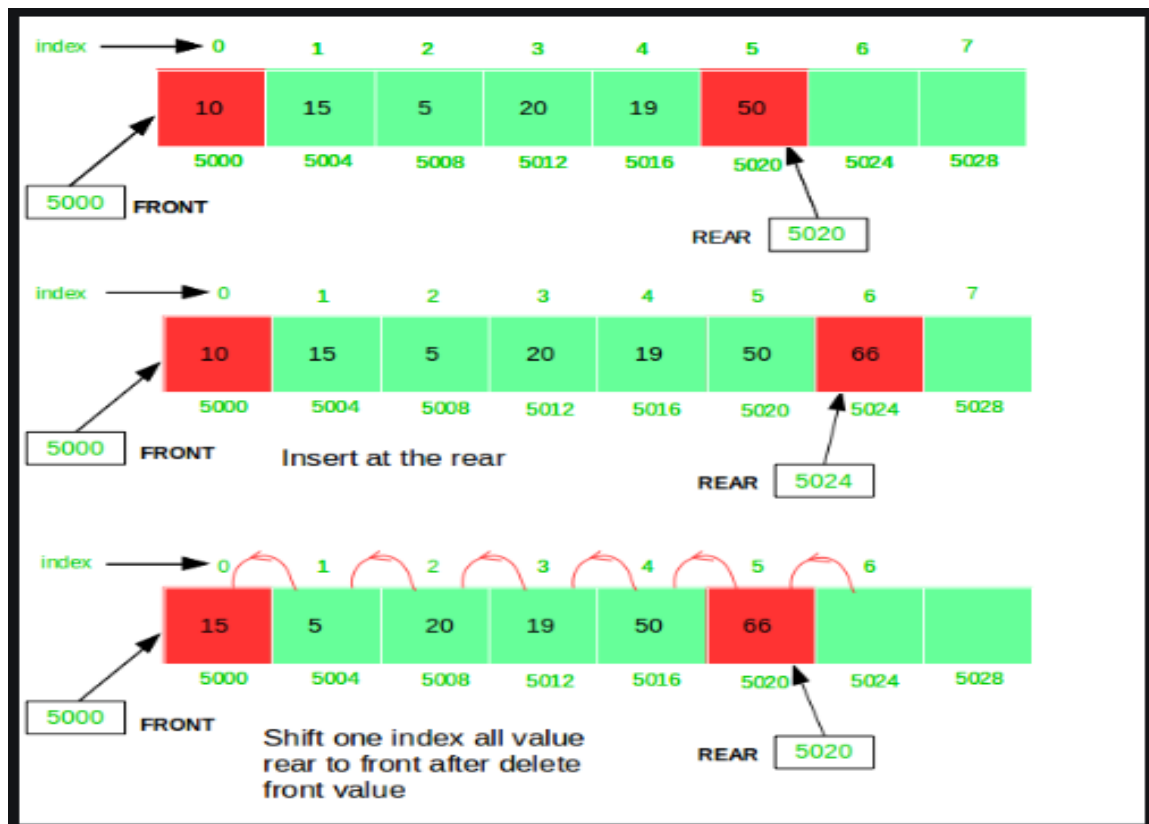
- **Task scheduling** in operating systems
- **Data transfer** in network communication
- **Simulation** of real-world systems (e.g., waiting lines)
- **Priority queues** for event processing queues for event processing



Representation of Queue using Array

To implement a queue using a simple array,

- I. Create an array of size **n** and
 - II. take two variables **front** and **rear** which are initialized as 0 and -1 respectively
 - III. **rear** is the index up to which the elements are stored in the array including the rear index itself. We mainly add an item by incrementing it.
 - IV. **front** is the index of the first element of the array. We mainly remove the element at this index in Dequeue operation.
-
- **Enqueue:** Addition of an element to the queue. Adding an element will be performed after checking whether the queue is full or not. If $\text{rear} == \text{capacity} - 1$, then it is said to be an Overflow condition as the array is full. If array is not full. then store the element at $\text{arr}[\text{rear}]$ and increment rear by 1 .
 - **Dequeue:** Removal of an element from the queue. An element can only be deleted when there is at least an element to delete. If $\text{rear} == -1$, then it is said to be an Underflow condition as the array is empty. If the element at $\text{arr}[\text{front}]$ can be deleted then move all the remaining elements to the left by one position in order to maintain the FIFO (the first element must be deleted).
 - **Front:** Get the front element from the queue i.e. $\text{arr}[\text{front}]$ if the queue is not empty.
 - **Display:** Print all elements of the queue. If the queue is non-empty, traverse and print all the elements from the index front to rear.



```
#include <bits/stdc++.h>

using namespace std;

struct Queue {
    int front, rear, capacity;
    int* queue;

    // Constructor to initialize the queue
    Queue(int c)
    {
        front = 0;
        rear = -1;
        capacity = c;
        queue = new int[c];
    }

    // Destructor to free the allocated memory
    ~Queue() { delete[] queue; }
```

// Function to insert an element at the rear of the queue

```
void queueEnqueue(int data)
```

```
{
```

```
    // Check if the queue is full
```

```
    if (rear == capacity - 1) {
```

```
        printf("\nQueue is full\n");
```

```
        return;
```

```
    }
```

```
    // Insert element at the rear
```

```
    queue[++rear] = data;
```

```
}
```

// Function to delete an element from the front

// of the queue

```
void queueDequeue()
```

```
{
```

```
    // If the queue is empty
```

```
    if (front > rear) {
```

```
        printf("\nQueue is empty\n");
```

```
        return;
```

```
    }
```

```
    // Shift all elements from index 1 till rear to
```

```
    // the left by one
```

```
    for (int i = 0; i < rear; i++) {
```

```
        queue[i] = queue[i + 1];
```

```
    }
```

```
    // Decrement rear
```

```
    rear--;
```

```
}
```

// Function to print queue elements

```
void queueDisplay()
```

```
{
    if (front > rear) {
        printf("\nQueue is Empty\n");
        return;
    }
    // Traverse front to rear and print elements
    for (int i = front; i <= rear; i++) {
        printf(" %d <-- ", queue[i]);
    }
    printf("\n");
}

// Function to print the front of the queue
void queueFront()
{
    if (rear == -1) {
        printf("\nQueue is Empty\n");
        return;
    }
    printf("\nFront Element is: %d\n", queue[front]);
}

};

// Driver code
int main(void)
{
    // Create a queue of capacity 4
    Queue q(4);
    // Print queue elements
    q.queueDisplay();
    // Insert elements in the queue
    q.queueEnqueue(20);
```

```
q.queueEnqueue(30);
q.queueEnqueue(40);
q.queueEnqueue(50);
// Print queue elements
q.queueDisplay();
// Insert element in the queue
q.queueEnqueue(60);
// Print queue elements
q.queueDisplay();
// Dequeue elements
q.queueDequeue();
q.queueDequeue();
printf("\nAfter two node deletions\n");
// Print queue elements
q.queueDisplay();

printf("\nAfter one insertion\n");
q.queueEnqueue(60);
// Print queue elements
q.queueDisplay();
// Print front of the queue
q.queueFront();
return 0;
}
```

Output

Queue is Empty

20 <-- 30 <-- 40 <-- 50 <--

Queue is full

20 <-- 30 <-- 40 <-- 50 <--

After two node deletions

40 <-- 50 <--

After one insertion

40 <-- 50 <-- 60 <--

Front Element...

Representation of Queue using Linked List

Follow the below steps to solve the problem:

- A. Create a class QNode with data members integer data and QNode* next
 - a. A parameterized constructor that takes an integer x value as a parameter and sets data equal to x and next as NULL
- B. Create a class Queue with data members QNode front and rear
- C. Enqueue Operation with parameter x:
 - a. Initialize QNode* temp with data = x
 - b. If the rear is set to NULL then set the front and rear to temp and return(Base Case)
 - c. Else set rear next to temp and then move rear to temp
- D. Dequeue Operation:
 - a. If the front is set to NULL return(Base Case)
 - b. Initialize QNode temp with front and set front to its next

- c. If the front is equal to NULL then set the rear to NULL
- d. Delete temp from the memory

```
// C++ program to implement the queue data structure using
// linked list
#include <bits/stdc++.h>
using namespace std;

// Node class representing a single node in the linked list
class Node {
public:
    int data;
    Node* next;
    Node(int new_data)
    {
        this->data = new_data;
        this->next = nullptr;
    }
};

// Class to implement queue operations using a linked list
class Queue {

    // Pointer to the front and the rear of the linked list
    Node *front, *rear;

public:
    // Constructor to initialize the front and rear
    Queue() { front = rear = nullptr; }

    // Function to check if the queue is empty
    bool isEmpty()
    {
        // If the front and rear are null, then the queue is
        // empty, otherwise it's not
        if (front == nullptr) {
            return true;
        }
        return false;
    }

    // Function to add an element to the queue
    void enqueue(int new_data) {

        // Create a new linked list node
```

```
Node* new_node = new Node(new_data);

// If queue is empty, the new node is both the front
// and rear
if (this->isEmpty()) {
    front = rear = new_node;
    return;
}

// Add the new node at the end of the queue and
// change rear
rear->next = new_node;
rear = new_node;
}

// Function to remove an element from the queue
void dequeue() {

    // If queue is empty, return
    if (this->isEmpty()) {
        cout << "Queue Underflow\n";
        return;
    }

    // Store previous front and move front one node
    // ahead
    Node* temp = front;
    front = front->next;

    // If front becomes nullptr, then change rear also
    // to nullptr
    if (front == nullptr)
        rear = nullptr;

    // Deallocate memory of the old front node
    delete temp;
}

// Function to get the front element of the queue
int getFront() {

    // Checking if the queue is empty
    if (this->isEmpty()) {
        cout << "Queue is empty\n";
        return INT_MIN;
    }
}
```

```
        return front->data;
    }

// Function to get the rear element of the queue
int getRear() {

    // Checking if the queue is empty
    if (this->isEmpty()) {
        cout << "Queue is empty\n";
        return INT_MIN;
    }
    return rear->data;
}
};

// Driver code to test the queue implementation
int main()
{
    Queue q;

    // Enqueue elements into the queue
    q.enqueue(10);
    q.enqueue(20);

    // Display the front and rear elements of the queue
    cout << "Queue Front: " << q.getFront() << endl;
    cout << "Queue Rear: " << q.getRear() << endl;

    // Dequeue elements from the queue
    q.dequeue();
    q.dequeue();

    // Enqueue more elements into the queue
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);

    // Dequeue an element from the queue
    q.dequeue();

    // Display the front and rear elements of the queue
    cout << "Queue Front: " << q.getFront() << endl;
    cout << "Queue Rear: " << q.getRear() << endl << endl;

    return 0;
}
```

Output

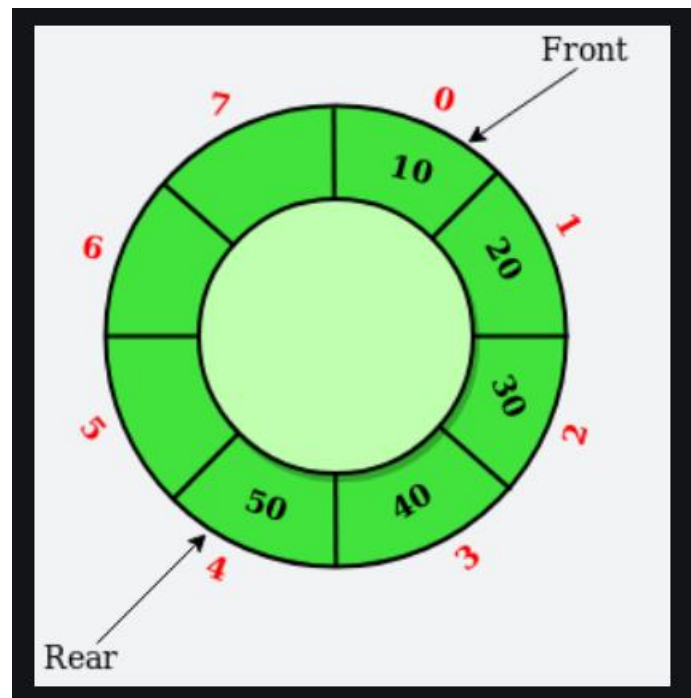
```
Queue Front: 10  
Queue Rear: 20  
Queue Front: 40  
Queue Rear: 50
```

G. Circular Queue

. A Circular Queue is an extended version of a normal queue where the last element of the queue is connected to the first element of the queue forming a circle.

The operations are performed based on FIFO (First In First Out) principle. It is also called '**Ring Buffer**'.

In a normal Queue, we can insert elements until queue becomes full. But once queue becomes full, we cannot insert the next element even if there is a space in front of queue.

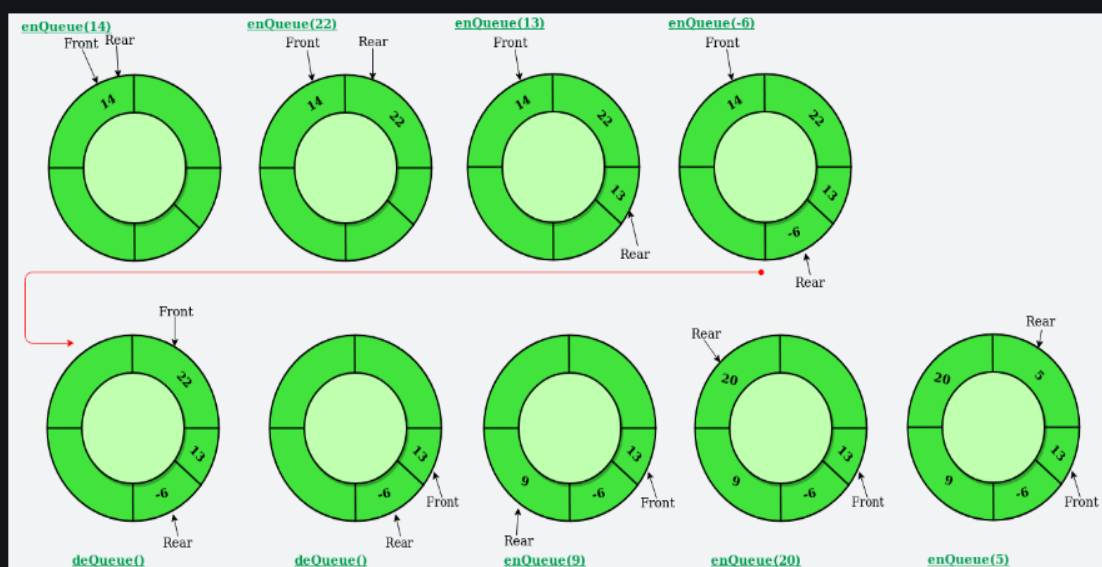


Operations on Circular Queue:

- A. **Front:** Get the front item from the queue.
- B. **Rear:** Get the last item from the queue.
- C. **enqueue(value)** This function is used to insert an element into the circular queue. In a circular queue, the new element is always inserted at the rear position.
 - a. Check whether the queue is full – [i.e., the rear end is in just before the front end in a circular manner].
 - b. If it is full then display Queue is full.
 - i. If the queue is not full then, insert an element at the end of the queue.
- D. **dequeue()** This function is used to delete an element from the circular queue. In a circular queue, the element is always deleted from the front position.
 - a. Check whether the queue is Empty.
 - b. If it is empty then display Queue is empty.
 - i. If the queue is not empty, then get the last element and remove it from the queue.

Illustration of Circular Queue Operations:

Follow the below image for a better understanding of the enqueue and dequeue operations.



Working of Circular queue operations

Implement Circular Queue using Array:

1. Initialize an array queue of size **n**, where n is the maximum number of elements that the queue can hold.
2. Initialize two variables front and rear to -1.
3. **Enqueue:** To enqueue an element **x** into the queue, do the following:
 - Increment rear by 1.
 - If **rear** is equal to n, set **rear** to 0.
 - If **front** is -1, set **front** to 0.
 - Set queue[rear] to x.
4. **Dequeue:** To dequeue an element from the queue, do the following:
 - Check if the queue is empty by checking if **front** is -1.
 - If it is, return an error message indicating that the queue is empty.
 - Set **x** to queue[front].
 - If **front** is equal to **rear**, set **front** and **rear** to -1.
 - Otherwise, increment **front** by 1 and if **front** is equal to n, set **front** to 0.
 - Return x.

```
// C or C++ program for insertion and
// deletion in Circular Queue
#include<bits/stdc++.h>
using namespace std;
```

```
class Queue
{
    // Initialize front and rear
    int rear, front;

    // Circular Queue
    int size;
    int *arr;
public:
    Queue(int s)
    {
        front = rear = -1;
        size = s;
        arr = new int[s];
    }
}
```

```
void enqueue(int value);
int dequeue();
void displayQueue();
};

/* Function to create Circular queue */
void Queue::enqueue(int value)
{
    if ((front == 0 && rear == size-1) ||
        ((rear+1) % size == front))
    {
        printf("\nQueue is Full");
        return;
    }

    else if (front == -1) /* Insert First Element */
    {
        front = rear = 0;
        arr[rear] = value;
    }

    else if (rear == size-1 && front != 0)
    {
        rear = 0;
        arr[rear] = value;
    }

    else
    {
        rear++;
        arr[rear] = value;
    }
}

// Function to delete element from Circular Queue
int Queue::dequeue()
{
    if (front == -1)
```

```
{
    printf("\nQueue is Empty");
    return INT_MIN;
}

int data = arr[front];
arr[front] = -1;
if (front == rear)
{
    front = -1;
    rear = -1;
}
else if (front == size-1)
    front = 0;
else
    front++;

return data;
}

// Function displaying the elements
// of Circular Queue
void Queue::displayQueue()
{
    if (front == -1)
    {
        printf("\nQueue is Empty");
        return;
    }
    printf("\nElements in Circular Queue are: ");
    if (rear >= front)
    {
        for (int i = front; i <= rear; i++)
            printf("%d ", arr[i]);
    }
    else
    {
        for (int i = front; i < size; i++)
            printf("%d ", arr[i]);
    }
}
```



```
        for (int i = 0; i <= rear; i++)
            printf("%d ", arr[i]);
    }
}

/* Driver of the program */
int main()
{
    Queue q(5);

    // Inserting elements in Circular Queue
    q.enqueue(14);
    q.enqueue(22);
    q.enqueue(13);
    q.enqueue(-6);

    // Display elements present in Circular Queue
    q.displayQueue();

    // Deleting elements from Circular Queue
    printf("\nDeleted value = %d", q.dequeue());
    printf("\nDeleted value = %d", q.dequeue());

    q.displayQueue();

    q.enqueue(9);
    q.enqueue(20);
    q.enqueue(5);

    q.displayQueue();

    q.enqueue(20);
    return 0;
}
```

Output

```
Elements in Circular Queue are: 14 22 13 -6  
Deleted value = 14  
Deleted value = 22  
Elements in Circular Queue are: 13 -6  
Elements in Circular Queue are: 13 -6 9 20 5  
Queue is Full
```

Applications of Circular Queue:

1. **Memory Management:** The unused memory locations in the case of ordinary queues can be utilized in circular queues.
2. **Traffic system:** In computer-controlled traffic system, circular queues are used to switch on the traffic lights one by one repeatedly as per the time set.
3. **CPU Scheduling:** Operating systems often maintain a queue of processes that are ready to execute or that are waiting for a particular event to occur.